Same Merges, Different Order: BPE Merge Sequence Shapes Language Modeling but Not Fine-Tuned Classification

NeuriCo

Abstract

Byte-pair encoding (BPE) tokenizers are trained once and then frozen into every model built on them, yet the order of their merges is almost never varied. We ask whether merge order matters, both the order in which merges are *learned* and the priority with which they are *applied*. We train six byte-level BPE tokenizers with the same 8,192-entry vocabulary size on the same corpus; they differ only in merge order. Two of them keep the exact vocabulary of standard frequency BPE and change only merge priority. For each tokenizer we pre-train identical 23M-parameter language models on the same 200 MB of general and biomedical text, and then fine-tune them on three classification tasks. The answer depends on what we measure. For language modeling, merge order clearly matters (all 10 comparisons with Holm-adjusted $p \leq 0.032$): the four non-frequency orders raise bits-per-byte by 1.8–5.8% on general text and by 2.8–9.8% on biomedical text. Even reordering the *same* merges costs 1.8–5.7%, because short merges fire first and block long ones, leaving 59–79% of the vocabulary unused. Compression explains this effect: bytes-per-token perfectly rank-predicts general-text bits-per-byte across tokenizers (Spearman $\rho = -1.0$). For fine-tuned classification, merge order shows no detectable effect: all tokenizers are within -0.8% to $+1.6\%$ of standard BPE, and none of the 15 comparisons is significant. A domain-weighted merge order gives a small domain gain (0.8% bits-per-byte) and no classification gain. Merge order is therefore not a cosmetic detail for generative modeling, but small fine-tuned classifiers are robust to it.

1 Introduction

A BPE tokenizer is a list of merges [Gage, 1994, Sennrich et al., 2016]. Training adds merges to this list one at a time, each time choosing the most frequent adjacent pair. At encoding time, the list order becomes a priority: the tokenizer repeatedly applies the highest-priority merge that fits [Radford et al., 2019]. The tokenizer is then frozen into every model built on top of it. Because the frequency rule is so standard, merge order is almost never treated as a design choice. Our main question is simple: **does the order of BPE merges change the quality of a model built on the tokenizer?**

The question matters in practice. Practitioners change merge order whenever they re-weight a tokenizer corpus toward a domain, add domain-specific tokens, or merge vocabularies from different sources. If quality is sensitive to merge order, these changes need care. If models are robust to it, merge order can be changed freely, for example for domain adaptation. Prior work compares tokenizer *algorithms* [Bostrom and Durrett, 2020, Ali et al., 2024], vocabulary sizes and compression [Gallé, 2019, Goldman et al., 2024], intrinsic efficiency measures [Zouhar et al., 2023], stochastic segmentation [Provilkov et al., 2020, Kudo, 2018], and domain vocabularies [Beltagy et al., 2019, Gu et al., 2021]. However, these comparisons change many things at once. We are not aware of a controlled study that varies *only* merge order while holding data, vocabulary size, model, and train-

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

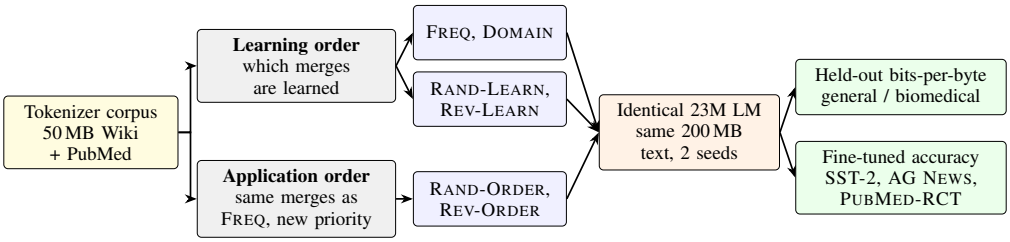


Figure 1: Study design. We train six byte-level BPE tokenizers (8,192 entries each) that differ only in merge order. Four change the *learning order*, which changes the vocabulary. Two keep FREQ's exact merges and change only the *application order*, i.e., merge priority at encoding time. Every tokenizer is paired with an identical language model trained on the same text, and we evaluate it with tokenizer-independent bits-per-byte and with fine-tuned classification.

ing text fixed. Prior work also does not separate *which* merges are learned from the *priority* with which they are applied.

We address this gap with a controlled from-scratch study (figure 1). We train six byte-level BPE tokenizers with 8,192 vocabulary entries on the same corpus, and we vary merge order in two ways. Four tokenizers change the *learning order*, and hence the vocabulary: standard frequency BPE (FREQ), domain-weighted frequency (DOMAIN), random choice among frequent pairs (RAND-LEARN), and least-frequent-first (REV-LEARN). Two tokenizers change only the *application order*: they keep FREQ's exact merges and vocabulary, but apply them in a random (RAND-ORDER) or reversed (REV-ORDER) priority. For each tokenizer, we pre-train identical 23M-parameter GPT-style models on the same 200 MB of Wikipedia and PubMed text. We measure tokenizer-independent bits-per-byte (BPB), and then fine-tune each model on SST-2, AG NEWS, and PUBMED-RCT sentence classification.

The results split cleanly by metric. For language modeling, merge order clearly matters. On general text, FREQ beats every alternative. The four non-frequency orders raise BPB by 1.8–5.8% on general text and by 2.8–9.8% on biomedical text, and all 10 BPB comparisons against FREQ are significant after Holm correction. The same-vocabulary controls are the most telling. RAND-ORDER and REV-ORDER lose 1.8–5.7% BPB with *exactly* the same merges as FREQ, because short merges that fire early consume their inputs and block long, frequent tokens. For example, REV-ORDER splits "hypertension" into eight pieces, while FREQ keeps it as one token. Compression explains the whole ranking: bytes-per-token rank-predicts general-text BPB perfectly (Spearman $\rho = -1.0$, $n = 6$). For fine-tuned classification, in contrast, every tokenizer lands within -0.8% to $+1.6\%$ of FREQ, and none of the 15 comparisons is significant. Even a nearly character-level tokenizer (REV-LEARN) is within 1% of FREQ on every task. Domain weighting buys a real but small domain gain: 0.8% BPB on PubMed, at a cost of 0.4% on general text.

In summary, we make the following contributions:

- We design a controlled study that isolates BPE merge order, and we separate learning order (which merges exist) from application order (which merges fire first) through same-vocabulary controls.
- We show that merge order significantly affects language modeling (+1.8–9.8% BPB) and that pure reordering of an identical vocabulary costs up to 5.7%, leaving 59–79% of the vocabulary unused.
- We identify compression as the mechanism: bytes-per-token rank-predicts BPB perfectly on general text ($\rho = -1.0$) and strongly on biomedical text ($\rho = -0.94$), while Rényi efficiency does not.
- We find that fine-tuned classification is robust to merge order: our confidence intervals rule out effects larger than about 1.7% on AG NEWS and PUBMED-RCT, and about 3.5% on SST-2.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 describes the tokenizers, models, and evaluation. Section 4 presents our results, and section 5 discusses their implications and limitations.

2 Related Work

Comparing tokenizers. BPE started as a compression algorithm [Gage, 1994] and was adapted to subword segmentation for neural machine translation [Sennrich et al., 2016]. GPT-2 later made it byte-level [Radford et al., 2019]. Several studies compare tokenizer families. Bostrom and Durrett [2020] find that unigram language-model tokenization [Kudo, 2018] matches or beats BPE for pre-training. Ali et al. [2024] show that tokenizer choice affects the downstream performance and training cost of multilingual LLMs. Schmidt et al. [2024] argue that tokenization is more than compression, because tokenizers with similar compression can give different downstream results. These studies vary the algorithm, the vocabulary, and often the training data together. Unlike them, we fix the algorithm, the corpus, and the vocabulary size, and we vary only merge order. Our same-vocabulary controls go one step further and fix the vocabulary itself.

Compression and intrinsic measures. Gallé [2019] link the success of BPE to shorter sequences. Goldman et al. [2024] show that compression correlates with downstream quality, especially for generation. Zouhar et al. [2023] propose Rényi efficiency as a better predictor of translation quality than compression. We test both measures on tokenizers that differ only in merge order. Bytes-per-token rank-predicts BPB perfectly in our setting, while Rényi efficiency is a much weaker predictor. We also find that the link between compression and quality holds for language modeling but not for fine-tuned classification.

Stochastic and alternative segmentation. BPE-dropout [Provilkov et al., 2020] and subword regularization [Kudo, 2018] randomly perturb segmentation during training to improve robustness. These methods change segmentation at training time while keeping the standard segmentation at test time. Our RAND-ORDER and REV-ORDER tokenizers instead fix a different, deterministic merge priority for both training and testing. At the extreme, our REV-LEARN tokenizer becomes almost character-level, which connects to byte-level models such as ByT5 [Xue et al., 2022]. Unlike ByT5, we keep the model and context length fixed, so a shorter-unit tokenizer sees less text per window.

Domain-specific vocabularies. Domain models such as SciBERT [Beltagy et al., 2019] and PubMedBERT [Gu et al., 2021] build vocabularies from in-domain text, and Gururangan et al. [2020] show that continued domain pre-training helps. These works change the vocabulary and the pre-training data together. Our DOMAIN tokenizer changes only merge order: it up-weights biomedical counts in the same mixed corpus. This isolates how much of a domain gain comes from the tokenizer alone.

3 Method

We first define the two kinds of merge order (section 3.1). We then describe our six tokenizers (section 3.2), the data and models (section 3.3), and the evaluation (section 3.4).

3.1 Two kinds of merge order

Learning order. Let $\mathcal{W}$ be the multiset of pre-tokens in the tokenizer corpus, each with count $c(w)$ and each split into a sequence of symbols, initially bytes. At step t, let $C_t(a, b)$ be the number of times the adjacent pair (a, b) occurs, weighted by $c(w)$. Standard BPE adds the merge $m_t = \arg\max_{(a,b)} C_t(a, b)$, replaces each occurrence of (a, b) with a new symbol ab, and repeats. We replace the $\arg\max$ with a general *selection rule* σ, which picks a pair from the current counts (algorithm 1). The rule decides *which* merges end up in the vocabulary.

Application order. At encoding time, a BPE tokenizer assigns each merge a rank $\pi(m)$. Given a pre-token, it repeatedly applies the lowest-rank merge that matches any adjacent pair, until no merge applies. Standard BPE uses the learning order as the rank, $\pi(m_t) = t$. Any other rank gives a different segmentation with the *same* vocabulary. We only use ranks that respect dependencies: a merge that creates a symbol always ranks before every merge that uses that symbol as input. This keeps every merge reachable in principle. In practice, many merges never fire, because a short merge applied early can consume the bytes that a longer merge needs (section 4.1).

Algorithm 1 BPE training with a general selection rule

Input: Pre-token counts $c(w)$; number of merges K; selection rule σ; minimum pair count τ
Output: Ordered merge list $(m_1, \ldots, m_K)$

```
Split every pre-token w into bytes; V ← the 256 byte symbols
for t = 1, ..., K do
    C_t(a, b) ← Σ_w c(w) · #{(a, b) adjacent in w}        ▷ updated incrementally
    m_t ← σ(C_t)        ▷ FREQ: arg max; REV-LEARN: arg min over C_t ≥ τ
    Replace each occurrence of m_t = (a, b) by the new symbol ab; V ← V ∪ {ab}
return (m_1, ..., m_K)        ▷ rank π(m_t) = t unless reordered
```

Table 1: The six merge-order conditions. All tokenizers have 8,192 entries and are trained on the same 50 MB corpus. The application-order tokenizers share FREQ's merges and vocabulary exactly.

Tokenizer	Family	Rule
FREQ	standard	Merge the most frequent pair (standard BPE).
DOMAIN	learning order	Most frequent pair, with PubMed pre-token counts $\times 8$.
RAND-LEARN	learning order	Uniformly random pair among pairs with count ≥ 50.
REV-LEARN	learning order	Least frequent pair among pairs with count ≥ 50.
RAND-ORDER	application order	FREQ's merges; random dependency-respecting priority.
REV-ORDER	application order	FREQ's merges; dependency-respecting reversed priority (latest-learned first).

3.2 Six tokenizers

Table 1 lists our tokenizers. All of them are byte-level BPE with GPT-2 pre-tokenization [Radford et al., 2019] and 8,192 vocabulary entries (256 bytes plus 7,936 merges). We train them with our own BPE implementation on a 50 MB tokenizer corpus: 37.5 MB of WIKITEXT-103 and 12.5 MB of PubMed abstracts, keeping pre-tokens with count ≥ 2. To check our trainer, we compare its FREQ merges with the HuggingFace `tokenizers` `BpeTrainer`. The merge sets overlap 100% at 500, 1k, 4k, and all merges. We export every tokenizer to HuggingFace `tokenizers`, whose BPE model applies merges by rank.

Learning-order family. FREQ is standard BPE. DOMAIN multiplies the counts of PubMed pre-tokens by 8 before training, which favors biomedical merges. RAND-LEARN picks a pair uniformly at random among pairs with count ≥ 50. REV-LEARN picks the *least* frequent pair among pairs with count ≥ 50. The threshold $\tau = 50$ keeps both rules from merging pairs that occur only a handful of times.

Application-order family. RAND-ORDER and REV-ORDER keep FREQ's exact merge list and vocabulary, and change only the rank π. RAND-ORDER draws a random dependency-respecting order. REV-ORDER builds a dependency-respecting order that puts the latest-learned merges first whenever their inputs are available. These two tokenizers isolate merge sequence in its pure form: any difference from FREQ comes from priority alone.

Robustness tokenizers. To check that our random results do not depend on one lucky or unlucky draw, we train a second random draw (seed 1) of RAND-LEARN and RAND-ORDER, which we denote with the suffix `-T1`.

3.3 Data and models

Pre-training data. Every language model sees the same 200 MB of text: 150 MB of WIKITEXT-103 training articles [Merity et al., 2017] and 50 MB of PubMed-200k-RCT abstracts [Dernoncourt and Lee, 2017]. We remove 36,101 PubMed-200k sentences that also appear in the PubMed-20k validation or test sets, to avoid leakage into evaluation. For held-out language-model evaluation, we use the WIKITEXT-103 test split (60 articles, 1.28 MB) and 1,000 PubMed-20k-RCT test abstracts (1.8 MB).

Classification tasks. We fine-tune on two general tasks and one domain task. SST-2 [Socher et al., 2013] is binary sentiment classification; we train on 20k sentences and test on the 872-sentence val-

idation split. AG NEWS [Zhang et al., 2015] is 4-class topic classification; we use 20k training and 3k test items. PUBMED-RCT [Dernoncourt and Lee, 2017] is 5-class sentence-role classification in abstracts (e.g., METHODS, RESULTS); we use 20k training and 3k test sentences from PubMed-20k.

Model and pre-training. We use the same GPT-style decoder for every tokenizer: 6 layers, hidden size 512, 8 attention heads, a 512-token context, and tied input and output embeddings (23.3M parameters). We train for one epoch over the same 200 MB text with AdamW [Loshchilov and Hutter, 2019] (learning rate 10^{-3}, $\beta = (0.9, 0.95)$, weight decay 0.1), 200 warm-up steps followed by cosine decay, 32k tokens per step, bf16, and gradient clipping at 1.0. We use two pre-training seeds per tokenizer and one seed for each -T1 tokenizer, for 14 language models in total.

We hold the *text* fixed, not the compute. A tokenizer that compresses poorly turns the same text into more tokens, and so receives more gradient steps. We chose this design because it matches the natural question "which tokenizer should we use for this corpus?". It also favors the poor compressors, as we discuss in section 4.5.

Fine-tuning. We mean-pool the final hidden states and add a linear classification head, and we fine-tune all parameters. We use the same hyper-parameters for every tokenizer: 2 epochs, batch size 32, learning rate 2×10^{-4} with 6% warm-up and linear decay, and inputs truncated to 256 tokens. We do not tune per condition. Each (tokenizer, task) pair gets 2 fine-tuning seeds for each of the 2 pre-trained models, for 4 runs per pair and 84 fine-tuning runs in total.

3.4 Evaluation

Bits-per-byte. Per-token perplexity cannot be compared across tokenizers, because the tokens differ. We therefore report bits-per-byte, which normalizes by the text itself [Gao et al., 2020]:

$$\text{BPB} = \frac{\sum_i -\log_2 p_\theta(x_i \mid x_{<i})}{\sum_i |\text{bytes}(x_i)|}, \tag{1}$$

where x_i ranges over the predicted tokens and $|\text{bytes}(x_i)|$ is the UTF-8 length of token x_i. Lower is better. We report the relative change against FREQ, $\Delta = (\text{BPB} - \text{BPB}_{\text{FREQ}})/\text{BPB}_{\text{FREQ}}$.

Classification. We report accuracy and macro-F1, and the relative change in accuracy against FREQ.

Tokenizer intrinsics. On the held-out text, we measure: bytes per token; tokens per word; the share of the 1,000 most frequent words encoded as a single token; Rényi efficiency with $\alpha = 2.5$ [Zouhar et al., 2023]; vocabulary utilization, i.e., the share of vocabulary entries that appear in 5M tokens of encoded text; and segmentation agreement with FREQ.

Statistics. For BPB, we compute 95% confidence intervals of the relative change with a paired bootstrap over held-out documents (10k resamples) [Efron and Tibshirani, 1994], and test significance with a paired sign-flip permutation test on per-document BPB. For accuracy, we bootstrap and permute over test items, using per-item correctness averaged over the 4 runs. We apply the Holm correction [Holm, 1979] across all 25 comparisons against FREQ (10 for BPB and 15 for accuracy).

4 Results

We first show how merge order changes segmentation (section 4.1). We then show its effect on language modeling (section 4.2) and the compression mechanism behind it (section 4.3). Next we show that the effect does not reach fine-tuned classification (section 4.4). We close with robustness checks (section 4.5) and hypothesis verdicts (section 4.6).

4.1 Merge order changes segmentation drastically

Reordering the same merges breaks long tokens. Table 2 shows that RAND-ORDER and REV-ORDER, which have FREQ's exact vocabulary, pack 40–55% fewer bytes into each token than FREQ on general text (2.34 and 1.73 vs. 3.87 bytes per token). They also keep only 28% and 10% of the top-1k words whole, compared with 89% for FREQ. The reason is that merges compete for the same bytes. When a short merge is applied first, it consumes bytes that a longer, frequent merge needed, and the long merge can no longer fire. The word " hypertension" illustrates this:

Table 2: Tokenizer intrinsics on held-out general / biomedical text. "Top-1k whole" is the share of the 1,000 most frequent words encoded as a single token. "Vocab used" is the share of the 8,192 entries that occur in 5M encoded tokens. "Agree" is segmentation agreement with FREQ on general text. The application-order tokenizers share FREQ's vocabulary but use only 21–41% of it. Best value per column in **bold**.

	Bytes / token ↑		**Top-1k whole ↑**		**Rényi eff.**	**Vocab**	**Agree**	**LM train**
Tokenizer	Gen.	Dom.	Gen.	Dom.	Gen.	**used**	**w/ FREQ**	**tokens**
FREQ	**3.87**	4.11	**0.89**	0.92	0.52	**0.99**	1.00	**51.0M**
DOMAIN	3.65	**4.41**	0.82	**0.99**	0.54	**0.99**	0.88	52.5M
RAND-LEARN	1.82	1.78	0.09	0.10	0.34	0.91	0.25	110.0M
REV-LEARN	1.03	1.03	0.03	0.06	0.32	0.50	0.05	194.6M
RAND-ORDER	2.34	2.24	0.28	0.25	**0.55**	0.41	0.57	86.4M
REV-ORDER	1.73	1.71	0.10	0.10	0.44	0.21	0.28	116.1M

Table 3: Held-out bits-per-byte (lower is better; mean of 2 pre-training seeds) and relative change against FREQ with 95% paired-bootstrap CIs. All 10 comparisons are significant after Holm correction ($p_{\text{Holm}} = 0.032$ for DOMAIN on general text, 0.0025 for all others). The seed standard deviation is at most 0.002 on general text and 0.016 on biomedical text. Best value per column in **bold**.

	General (WIKITEXT-103)		**Biomedical** (PubMed)	
Tokenizer	BPB	Δ vs FREQ [95% CI]	BPB	Δ vs FREQ [95% CI]
FREQ	**1.348**	—	1.269	—
DOMAIN	1.353	+0.41% [0.23, 0.60]	**1.258**	**−0.83% [−1.04, −0.60]**
RAND-LEARN	1.386	+2.84% [2.39, 3.28]	1.305	+2.85% [2.57, 3.12]
REV-LEARN	1.426	+5.78% [5.11, 6.63]	1.394	+9.84% [9.49, 10.19]
RAND-ORDER	1.371	+1.76% [1.46, 2.07]	1.314	+3.58% [3.33, 3.83]
REV-ORDER	1.394	+3.43% [2.92, 4.00]	1.341	+5.71% [5.44, 5.98]

Tokenizer	**Segmentation of "hypertension"**
FREQ	`␣hypertension`
RAND-ORDER	`␣hy·pe·r·te·ns·io·n`
REV-ORDER	`␣·hy·pe·r·te·ns·io·n`
REV-LEARN	single characters

As a result, 59% (RAND-ORDER) and 79% (REV-ORDER) of the vocabulary never appear in 5M tokens of encoded text. The merges exist, but they are dead weight.

Changing what is learned matters even more. REV-LEARN learns only rare word fragments. It compresses to 1.03 bytes per token, so it is effectively a character-level tokenizer. RAND-LEARN sits in between, at 1.82 bytes per token. DOMAIN behaves as intended: it keeps 99% of the top-1k biomedical words whole (vs. 92% for FREQ) and improves biomedical compression by 7%, at a 6% loss on general text.

4.2 Merge order matters for language modeling

Every non-frequency order is significantly worse. Table 3 shows that, on general text, every alternative is worse than FREQ. The four non-frequency orders raise BPB by 1.8–5.8% on general text and by 2.8–9.8% on biomedical text. These gaps are large compared with run-to-run noise: the pre-training seed standard deviation is at most 0.002 BPB on general text, while the smallest non-frequency gap is 0.024. All 10 comparisons survive Holm correction.

Pure reordering costs almost as much as a random vocabulary. The application-order tokenizers isolate merge priority. RAND-ORDER loses 1.8% (general) and 3.6% (biomedical), and REV-ORDER loses 3.4% and 5.7%. REV-ORDER is *worse* than RAND-LEARN, which learns an entirely

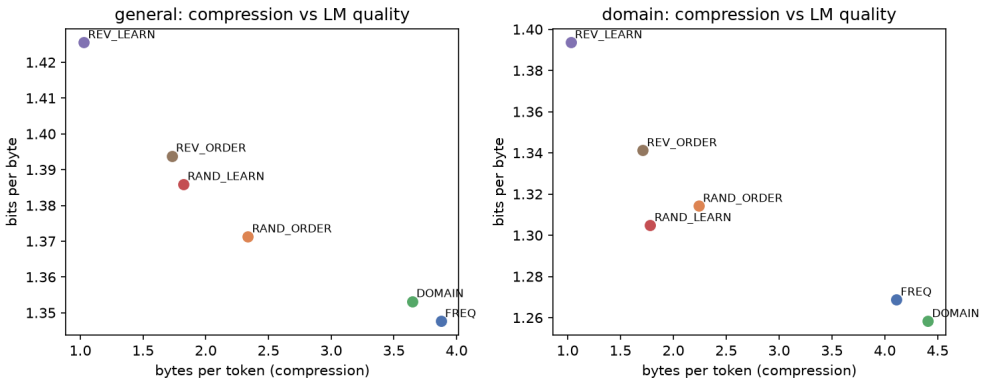


Figure 2: Compression predicts language-modeling quality. Each point is one tokenizer (mean of 2 seeds; labels use underscores, e.g., `REV_LEARN` is REV-LEARN). *(Left)*On general text, the rank of bytes-per-token matches the rank of BPB exactly (Spearman $\rho = -1.0$, exact $p = 0.003$). *(Right)*On biomedical text the relation is nearly monotone ($\rho = -0.94$, $p = 0.005$); RAND-LEARN and RAND-ORDER swap places, and DOMAIN overtakes FREQ by compressing better.

different, randomly chosen vocabulary. So merge priority is not a cosmetic detail of a merge list: in its pure form, merge order matters.

Reverse frequency is worst; domain weighting helps slightly. REV-LEARN is the worst tokenizer on both test sets (+5.8% and +9.8%). DOMAIN is the only tokenizer that beats FREQ anywhere. It improves biomedical BPB by 0.83% (95% CI [0.60, 1.04]) and costs 0.41% on general text.

4.3 Compression explains the language-modeling gap

Bytes per token rank-predicts BPB. Figure 2 plots BPB against compression. On general text, the ranks match exactly ($\rho = -1.0$, $n = 6$). On biomedical text, the relation is nearly perfect ($\rho = -0.94$). The only swap is between RAND-LEARN and RAND-ORDER, whose biomedical compression differs by 0.46 bytes per token while their BPB differs by only 0.009. Compression also explains DOMAIN: it is the best compressor of biomedical text and the best biomedical model.

Rényi efficiency does not. Rényi efficiency [Zouhar et al., 2023] ranks the tokenizers differently. RAND-ORDER has the *highest* general Rényi efficiency (0.55, table 2) but only the third-best BPB. Across the six tokenizers, Rényi efficiency correlates only weakly with BPB ($\rho = -0.71$, $p = 0.11$ on general text; $\rho = -0.49$, $p = 0.33$ on biomedical text). In our setting, plain compression is the better predictor.

Why compression matters here. With a fixed 512-token window, a poor compressor sees much less text: about 530 bytes per window for REV-LEARN vs. about 2,000 bytes for FREQ. It also spends capacity on predicting pieces inside words. The training curves in figure 4 (appendix A) are consistent with this. The per-byte training loss of all tokenizers overlaps for most of training, and the gaps open late, as FREQ and DOMAIN keep improving.

4.4 Merge order does not affect fine-tuned classification

All tokenizers reach the same accuracy. Table 4 and figure 3 show that every tokenizer lands within -0.8% to $+1.6\%$ of FREQ's accuracy. None of the 15 comparisons is significant (raw $p \geq 0.08$, Holm-adjusted $p = 1.0$). The confidence intervals are tight enough to be informative: they rule out effects larger than about 1.7% on AG NEWS and PUBMED-RCT, and about 3.5% on SST-2. On PUBMED-RCT, the four non-frequency tokenizers are all slightly below FREQ (-0.35% to -0.81%), but every interval includes zero.

Even a near-character tokenizer is competitive. REV-LEARN, which needs 5.0 tokens per word, is within 1% of FREQ on every task. This holds even on AG NEWS, where 23% of REV-LEARN's test inputs are truncated at 256 tokens (vs. 0.1% for FREQ). Its 5.8–9.8% BPB gap from pre-training simply does not carry over to these tasks.

Table 4: Fine-tuned accuracy (%, mean ± sd over 4 runs) and relative change against FREQ with 95% paired-bootstrap CIs over test items. None of the 15 comparisons is significant (raw $p \geq 0.08$; Holm-adjusted $p = 1.0$). Best mean per task in **bold**. Macro-F1 gives the same picture (table 8).

	SST-2		AG NEWS		PUBMED-RCT (domain)	
Tokenizer	Acc.	Δ [95% CI]	Acc.	Δ [95% CI]	Acc.	Δ [95% CI]
FREQ	80.2 ± 0.4	—	**89.3** ± 0.4	—	81.7 ± 0.4	—
DOMAIN	**81.5** ± 0.4	+1.57% [−0.43, 3.66]	88.9 ± 0.4	−0.48% [−1.14, 0.20]	**81.8** ± 0.3	+0.07% [−0.59, 0.75]
RAND-LEARN	79.7 ± 0.6	−0.64% [−3.34, 2.08]	88.8 ± 0.4	−0.58% [−1.41, 0.25]	81.4 ± 0.2	−0.35% [−1.24, 0.55]
REV-LEARN	80.8 ± 0.7	+0.72% [−1.72, 3.18]	88.7 ± 0.4	−0.71% [−1.57, 0.14]	81.1 ± 0.3	−0.75% [−1.69, 0.20]
RAND-ORDER	79.8 ± 0.7	−0.54% [−2.96, 1.93]	88.8 ± 0.2	−0.50% [−1.29, 0.28]	81.1 ± 0.5	−0.75% [−1.60, 0.09]
REV-ORDER	80.0 ± 0.9	−0.21% [−2.80, 2.53]	88.9 ± 0.4	−0.46% [−1.28, 0.37]	81.1 ± 0.2	−0.81% [−1.70, 0.07]

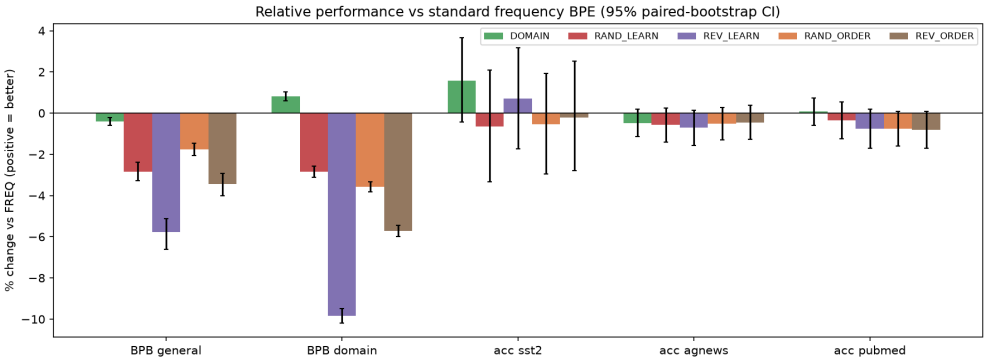


Figure 3: Relative change against FREQ with 95% paired-bootstrap CIs; positive means better, so BPB changes are sign-flipped. The language-modeling gaps (left two groups) are large and have tight intervals. The fine-tuning differences (right three groups) are small, and every interval covers zero.

Domain weighting does not help the domain task. DOMAIN improves PUBMED-RCT accuracy by only 0.07%. Its +1.6% on SST-2 is the largest point gain in the table, but it is not significant ($p = 0.13$, CI $[-0.43, 3.66]$).

4.5 Robustness

Random draw. Table 5 shows that a second random draw of each random tokenizer gives effects of the same size: +1.7–3.0% general BPB across all four random tokenizers, and accuracy within 0.7 points of the first draw. Our conclusions do not depend on the particular permutation.

Seeds. The pre-training seed standard deviation of BPB (≤ 0.016) is much smaller than the gaps between FREQ and the four non-frequency orders (≥ 0.024), and the fine-tuning standard deviation (≤ 0.9 points) is similar to the between-tokenizer differences.

Compute does not compensate. Because we hold text fixed, poor compressors get more tokens and more gradient steps: 1.7–3.8× the compute of FREQ (table 7, appendix A). REV-LEARN takes 5,939 steps, while FREQ takes 1,555. FREQ is still best on general text. An equal-compute comparison would therefore favor FREQ even more.

4.6 Hypothesis verdicts

Table 6 summarizes our initial hypotheses. The qualitative picture largely holds for BPB: FREQ wins on general text, random orders lose a few percent, and reverse-frequency is worst. However, the hypothesized effect sizes are too large. Reverse frequency loses 5.8–9.8% rather than 10–20%, and domain weighting gains 0.8% rather than 5–12%. None of the effects reaches fine-tuned accuracy.

Table 5: Robustness to the random draw. A second draw (-T1) of each random tokenizer, trained with pre-training seed 0, gives effects of the same size as the first draw. Δ is relative to FREQ seed 0. Accuracy (%) is the mean of 2 fine-tuning seeds on the seed-0 model; differences between draws are within 0.7 points on every task.

Tokenizer	Bytes/token	General ΔBPB	Domain ΔBPB	SST-2	AG NEWS	PUBMED-RCT
RAND-ORDER	2.34	+1.7%	+3.7%	80.1	88.8	81.1
RAND-ORDER -T1	2.19	+2.5%	+2.3%	79.5	88.6	81.1
RAND-LEARN	1.82	+3.0%	+3.4%	79.6	88.5	81.4
RAND-LEARN -T1	1.76	+2.7%	+5.3%	79.0	89.0	81.5

Table 6: The four hypotheses we set out to test and their verdicts. The qualitative ordering largely holds for BPB, but the hypothesized effect sizes are too large, and none of the effects reaches fine-tuned accuracy.

Hypothesis	Observed	Verdict
H1: FREQ beats alternatives by 3–8% on general tasks	BPB: +1.8–5.8%; accuracy: $\leq$1.6%, n.s.	Partly (BPB only)
H2: DOMAIN beats FREQ by 5–12% on its domain	BPB: 0.8%; accuracy: 0.07%, n.s.	Not supported
H3: Random order is only 2–5% worse	BPB: +1.8–3.6%; accuracy: $\approx$0.5%, n.s.	Roughly (BPB)
H4: Reverse-frequency degrades most (10–20%)	Worst for BPB (+5.8% / +9.8%); not for accuracy	Direction only

5 Discussion

Does merge order matter? Yes, but through compression. Our results confirm that merge sequence matters for language modeling. It matters through segmentation quality, not because a model is sensitive to merge order as such. Every BPB gap in our study follows how many bytes each token carries. The same-vocabulary controls make this point most clearly. RAND-ORDER and REV-ORDER have exactly FREQ's merges, so they *could* produce the same long tokens. Their different priority prevents this, leaving 59–79% of the vocabulary unused. A merge list is therefore not just a set of tokens: the order is part of the tokenizer, and changing it can quietly turn an 8k vocabulary into a 2k one. Anyone who edits a merge list, for example by inserting domain merges at the top, should check vocabulary utilization and compression afterward.

Why is fine-tuning robust? With 20k labeled examples, a 23M-parameter model fine-tuned from any of our language models reaches the same accuracy within about 1%. We see two likely reasons. First, short-text classification depends on lexical and semantic cues that the model can recover from any reasonable segmentation, even a nearly character-level one. Second, plentiful labeled data lets fine-tuning make up for weaker pre-training. This is consistent with Goldman et al. [2024], who find that compression matters more for generation than for classification. It also suggests that work like Schmidt et al. [2024], which finds downstream differences between tokenizers with similar compression, may need generative or low-data tasks to see large effects.

Why is domain weighting so weak? Up-weighting PubMed counts by 8 kept 99% of the top-1k biomedical words whole, up from 92%, and improved biomedical compression by 7%. This bought only 0.8% biomedical BPB, at a cost of 0.4% general BPB, and no classification gain. The reason is that the standard tokenizer corpus is already 25% biomedical, so FREQ already covers most biomedical vocabulary. The hypothesized 5–12% domain advantage would likely require a much more under-represented domain or a much smaller vocabulary.

Practical implications. For small models fine-tuned on classification with plenty of labels, merge order is close to irrelevant, so practitioners can re-weight or reorder merges without measurable cost. For language modeling or generation, merge order matters, and the rule is simple: prefer the order that keeps common words whole and gives the best bytes per token on the target text, even when the vocabulary is the same.

5.1 Limitations

Scale. Our models have 23M parameters, are trained on 200 MB of text (51–195M tokens), and use a single vocabulary size (8,192). Larger models, longer training, or larger vocabularies may change the size of the effects. In particular, we do not know whether the BPB gaps grow or shrink with scale.

Equal-text design. Our conditions differ in tokens and compute per byte. We hold text fixed, which is the natural "same corpus" comparison. An equal-compute or equal-byte-context design would separate these factors further. Because the poor compressors received more compute, we expect such a design to widen, not close, the BPB gaps.

Tasks. Our three tasks are short-text classification with plentiful labels, which likely makes them insensitive to tokenization. Generation, few-shot prompting, spelling or arithmetic tasks, and low-resource fine-tuning may reveal effects that our tasks hide.

Seeds and domains. We use 2 pre-training seeds per tokenizer (1 for the `-T1` variants), one tokenizer corpus, and one target domain (biomedical).

Definitions of order. Our definitions of "random" and "reverse" are design choices: a count threshold of 50 for the learning rules, and dependency-respecting reorderings for application order. Other definitions, such as sampling pairs in proportion to frequency, could give milder degradation.

Fixed fine-tuning hyper-parameters. We use the same fine-tuning hyper-parameters for every tokenizer. This is fair, but it could slightly disadvantage tokenizers that produce longer sequences, for example through truncation at 256 tokens.

Broader impact. This study uses public datasets and small models, and it trains no system intended for deployment. Our findings may help practitioners avoid tokenizer edits that silently reduce compression and waste vocabulary, which in turn wastes compute. We do not foresee direct negative uses.

6 Conclusion

We ask whether the order of BPE merges, both learned and applied, affects the models built on a tokenizer. In a controlled study with six tokenizers, 14 language models, and 84 fine-tuning runs, the answer depends on the metric. For language modeling, merge order matters: the four non-frequency orders raise bits-per-byte by 1.8–9.8%, all significantly, and even a pure reordering of an identical vocabulary costs up to 5.7%. Compression fully explains this: the order that keeps common words whole wins. For fine-tuned classification at our scale, merge order does not measurably matter (at most 1.6%, not significant). Domain-weighted merge order gives a real but small 0.8% domain gain.

The key takeaway is that a merge list is an ordered object: its order decides which tokens are actually used, and so how well a language model can compress text.

Several follow-ups would test how far these findings extend: equal-compute and equal-byte-context controls; models with more than 100M parameters and 32k vocabularies; generation, few-shot, and low-data (100–1,000 example) evaluations; and a rarer target domain, such as chemistry or code, where domain-biased merges may help more.

References

Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Schulze Buschhoff, et al. Tokenizer choice for LLM training: Negligible or crucial? In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.

Iz Beltagy, Kyle Lo, and Arman Cohan. SciBERT: A pretrained language model for scientific text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3615–3620, 2019.

Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4617–4624, 2020.

Franck Dernoncourt and Ji Young Lee. PubMed 200k RCT: a dataset for sequential sentence classification in medical abstracts. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (IJCNLP)*, pages 308–313, 2017.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.

Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

Matthias Gallé. Investigating the effectiveness of BPE: The power of shorter sequences. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1375–1381, 2019.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

Omer Goldman, Avi Caciularu, Matan Eyal, Kris Cao, Idan Szpektor, and Reut Tsarfaty. Unpacking tokenization: Evaluating text compression and its correlation with model performance. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare*, 3(1):1–23, 2021.

Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8342–8360, 2020.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 66–75, 2018.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1882–1892, 2020.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

Craig W. Schmidt, Varshini Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. Tokenization is more than compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1631–1642, 2013.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.

Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 649–657, 2015.

Vilém Zouhar, Clara Meister, Juan Luis Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 5184–5207, 2023.

A Additional Results

Training curves. Figure 4 shows the per-byte training loss of the seed-0 models.

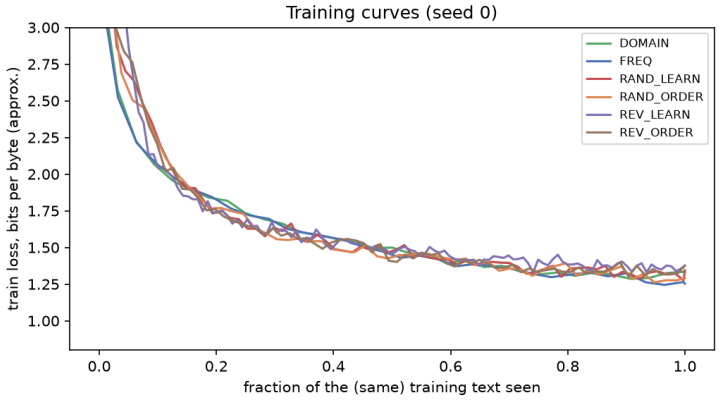


Figure 4: Approximate per-byte training loss (seed 0) against the fraction of the shared training text seen. The curves overlap for most of training; the gaps open late, as FREQ and DOMAIN keep improving. Labels use underscores (e.g., `REV_LEARN` is REV-LEARN).

Compute. Table 7 lists the training tokens, steps, and wall-clock time of each model. Because all models see the same text, poor compressors take more steps. Wall-clock times come from shared GPUs and are only a rough guide.

Table 7: Pre-training compute for seed 0. Compute is relative to FREQ (same model, so proportional to tokens). Poor compressors receive 1.7–3.8× more compute yet still reach worse BPB.

Tokenizer	Train tokens	Steps	Minutes	Rel. compute
FREQ	51.0M	1,555	24.3	1.00×
DOMAIN	52.4M	1,600	18.4	1.03×
RAND-LEARN	110.0M	3,358	33.2	2.16×
RAND-LEARN -T1	112.7M	3,438	56.8	2.21×
REV-LEARN	194.6M	5,939	32.4	3.82×
RAND-ORDER	86.4M	2,637	28.6	1.70×
RAND-ORDER -T1	91.3M	2,787	45.4	1.79×
REV-ORDER	116.1M	3,542	37.9	2.28×

Macro-F1. Table 8 reports macro-F1, which tells the same story as accuracy.

Table 8: Fine-tuned macro-F1 (%, mean ± sd over 4 runs) and relative change against FREQ. Best mean per task in **bold**. Mean test length in tokens is shown in parentheses, which reflects compression.

	SST-2			AG NEWS			PUBMED-RCT		
Tokenizer	F1	Δ	(len)	F1	Δ	(len)	F1	Δ	(len)
FREQ	80.2 ± 0.4	—	(30)	**89.3** ± 0.4	—	(70)	**75.4** ± 0.5	—	(37)
DOMAIN	**81.4** ± 0.4	+1.57%	(31)	88.9 ± 0.4	−0.48%	(74)	75.4 ± 0.2	−0.08%	(34)
RAND-LEARN	79.6 ± 0.6	−0.69%	(59)	88.8 ± 0.4	−0.58%	(131)	75.1 ± 0.6	−0.49%	(83)
REV-LEARN	80.7 ± 0.7	+0.66%	(104)	88.7 ± 0.4	−0.72%	(213)	74.8 ± 0.2	−0.91%	(140)
RAND-ORDER	79.7 ± 0.7	−0.54%	(47)	88.9 ± 0.2	−0.51%	(108)	74.9 ± 0.8	−0.67%	(67)
REV-ORDER	80.0 ± 0.9	−0.23%	(63)	88.9 ± 0.4	−0.47%	(139)	74.6 ± 0.4	−1.15%	(87)

Implementation details. Our BPE trainer keeps a dictionary of pre-token counts and updates pair counts incrementally after each merge. For the application-order tokenizers, we compute a dependency graph in which each merge points to the merges that use its output symbol, and we build a topological order of this graph by repeatedly choosing one merge whose inputs are already available: a uniformly random one for RAND-ORDER, and the latest-learned one for REV-ORDER. All experiments ran on shared NVIDIA RTX A6000 GPUs, using about 3 GB of memory per job; each language model took 18–57 minutes. We used PyTorch, HuggingFace `tokenizers`, NumPy, SciPy, pandas, and matplotlib.